

PCS-302 : Data Structures Lab

Queue Using a Linked List | Create, EnQueue, DeQueue, IsEmpty, Size

Problem Statement

Write an algorithm and a program to implement a queue using a linked list. The program should implement the following operations: Create(), EnQueue(k), DeQueue(), IsEmpty() and Size().

Example

```
create()      --> Queue created, empty

enqueue(10)   --> Queue: 10
enqueue(20)   --> Queue: 10 20
enqueue(30)   --> Queue: 10 20 30

size()        --> 3

dequeue()     --> removes 10 (front), Queue: 20 30
```

Algorithm

```
Create():
    front = NULL, rear = NULL, count = 0

EnQueue(k):
    create newNode with data = k, next = NULL
    if IsEmpty(): front = rear = newNode
    else: rear->next = newNode, rear = newNode
    count = count + 1

DeQueue():
    if IsEmpty(): print "Underflow", return
    temp = front
    front = front->next
    if front == NULL: rear = NULL
    free temp
    count = count - 1

IsEmpty():
    return (front == NULL)

Size():
    return count
```

C Program

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
```

```

};

struct Node *front = NULL;
struct Node *rear = NULL;
int count = 0;

void create() {
    front = NULL;
    rear = NULL;
    count = 0;
    printf("Queue created.\n");
}

int isEmpty() {
    if (front == NULL)
        return 1;
    else
        return 0;
}

void enqueue(int k) {
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
    newNode->data = k;
    newNode->next = NULL;

    if (isEmpty() == 1) {
        front = newNode;
        rear = newNode;
    } else {
        rear->next = newNode;
        rear = newNode;
    }
    count = count + 1;
    printf("%d enqueued to queue.\n", k);
}

void dequeue() {
    if (isEmpty() == 1) {
        printf("Queue Underflow. Cannot dequeue.\n");
        return;
    }
    struct Node *temp = front;
    printf("%d dequeued from queue.\n", temp->data);
    front = front->next;
    if (front == NULL) {
        rear = NULL;
    }
    free(temp);
    count = count - 1;
}

int size() {
    return count;
}

void printQueue() {

```

```

    struct Node *temp = front;
    if (isEmpty() == 1) {
        printf("Queue is empty.\n");
        return;
    }
    printf("Queue (front to rear): ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    create();

    enqueue(10);
    enqueue(20);
    enqueue(30);
    printQueue();

    printf("Size = %d\n", size());

    dequeue();
    printQueue();

    if (isEmpty() == 1)
        printf("Queue is empty.\n");
    else
        printf("Queue is not empty.\n");

    return 0;
}

```